

Introduction to Machine Learning — Lecture 1

Unit 2 · ML Fundamentals & Preprocessing | Class notes

Dataset used: `Titanic-Dataset.csv` | Library: `scikit-learn`

What this lecture covers

Section	Question it answers
1. Traditional programming vs ML	What is machine learning, really?
2. Supervised vs unsupervised	Does my data have an answer column?
3. Classification vs regression	What kind of answer am I predicting?
4. The ML pipeline	What are the steps, start to finish?
5. scikit-learn	Which tool do I use at each step?
6. Stage 1 — Data cleaning	How do I make the data usable?
7. Features and target	What is x, what is y?
8. Stage 2 — Train/test split	How do I test honestly?
9. Stage 3 — Encoding	How do text columns become numbers?

Setup

```
! pip3 install scikit-learn
```

```
import numpy as np
import pandas as pd
import sklearn
```

The CSV file must sit in the same folder as the notebook.

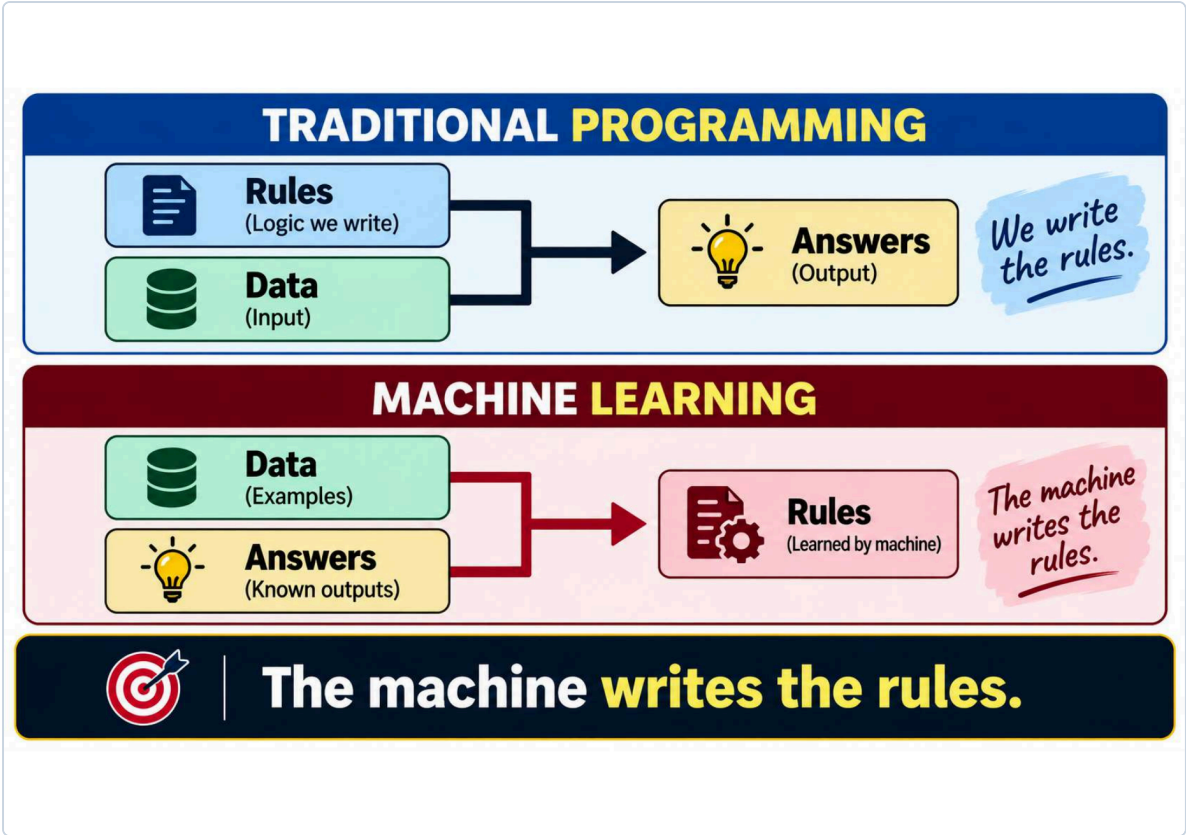
1. What is Machine Learning

Flow: Rules written by hand → rules learned from data.

In traditional programming, a person writes the rules. The computer takes those rules plus some data and produces answers. Every condition in an `if` statement was decided by a human being.

Machine learning turns that around. The computer is given the data *and* the answers that go with it, and it works out the rules by itself. Those learned rules are then applied to new data

the computer has never seen.



Traditional programming vs machine learning — what goes in, and what comes out

	Traditional programming	Machine learning
Input	Rules + Data	Data + Answers
Output	Answers	Rules
Who writes the rules	A programmer	The machine
Good when	The rules are few and known	The rules are too many, or nobody knows them

The one line to remember: in machine learning, the machine writes the rules.

✖ Practice 1

Q1. Gmail decides which mails go to the Spam folder.

- **Version A** — a rule is written: any mail containing the word "lottery" goes to Spam.
- **Version B** — millions of users have clicked "Report spam" over the years, and Gmail works out for itself what spam looks like.

Which is traditional programming, which is machine learning?

Answer. A is traditional programming — a person wrote the condition. B is machine learning — the mails and the past answers were given, and the rule was learned.

Q2. Your phone unlocks when it sees your face. Why would it be very hard to write that as a set of conditions by hand?

Answer. A face looks different with light, angle, glasses, a haircut, a beard. There is no small set of conditions that covers all of it — too many cases to write by hand, which is exactly when ML is useful.

2. Supervised vs Unsupervised

Flow: Does the data have an answer column, or not?

This is the first question to ask about any dataset, and it has a mechanical test: open the table and look for a column holding the answer you want to predict.

- **Supervised learning** — the answer column exists. On Titanic that column is `Survived`. The machine learns a rule that maps features → answer.
- **Unsupervised learning** — there is no answer column. The machine has to find natural groups (clusters) in the data instead of predicting a known answer.

TWO WAYS TO LEARN FROM DATA

SUPERVISED LEARNING

Learn from labelled data (answers are given)

Age	Fare	Survived
22	7.25	1
38	71.28	0
26	7.92	0
35	53.10	1
...	...	...

We learn a rule that maps
features → **answer**.



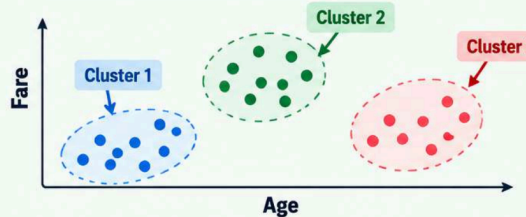
Predict whether a new passenger
survives or not.

UNSUPERVISED LEARNING

Find structure in unlabelled data (no answers)

Age	Fare
22	7.25
38	71.28
26	7.92
35	53.10
...	...

No answer
column at all.



We find natural groups (**clusters**)
in the data.



Same data, **different questions**, different learning style.

The same columns, with and without an answer column — that difference decides the learning style

	Supervised	Unsupervised
Answer column	Present (labelled data)	Absent (unlabelled data)
Goal	Predict the answer for a new row	Find structure / groups
Titanic example	Predict <code>Survived</code>	Group similar passengers with no <code>Survived</code> column
In this course	Unit 2 and Unit 3	Unit 4

Today is supervised.

✂ Practice 2

Q1. Swiggy has records of every order it has delivered, and each record says whether that delivery was late or on time. They want to predict this for a new order. Supervised or unsupervised?

Answer. Supervised — the answer column already exists in the records.

Q2. Spotify has the listening history of every user. Nobody has ever put users into categories. The company wants to find out whether natural groups of listeners exist. Supervised or unsupervised?

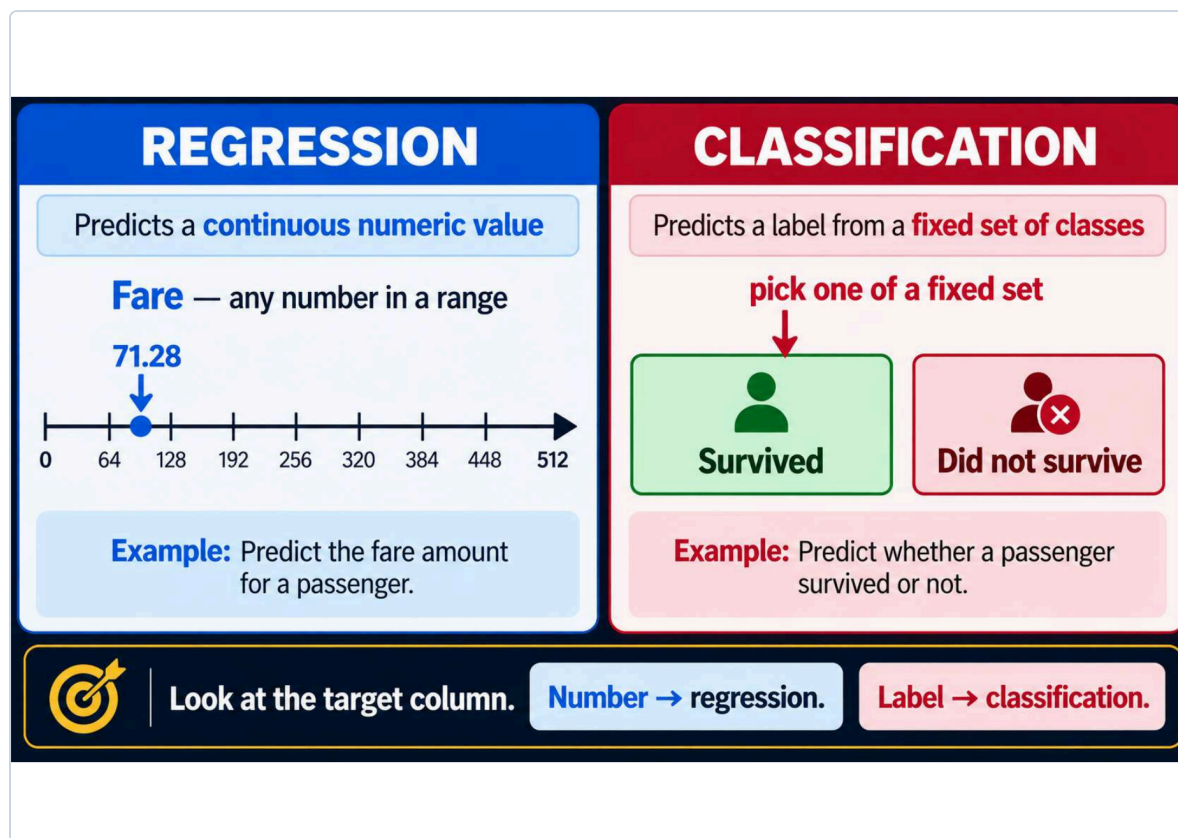
Answer. Unsupervised — there is no answer column, so the goal is to find groups, not to predict a known answer.

3. Classification vs Regression

Flow: Look at the target column, and only the target column.

Once you know the problem is supervised, the next split depends entirely on what the target column contains. Nothing else about the dataset matters for this decision.

- **Regression** — the target is a number in a range. Any value in that range is possible, including decimals. Predicting `Fare` is regression.
- **Classification** — the target is one of a fixed set of labels. Predicting `Survived` (0 or 1) is classification.



Number in a range → regression. One label out of a fixed set → classification.

```
df = pd.read_csv('Titanic-Dataset.csv')
df['Survived'].value_counts()
```

```
Survived
0      549
1      342
```

Only two values ever appear, so `Survived` is a classification target. The counts also give the baseline: $549 / 891 = 61.6\%$, which is what a model scores by always answering "did not survive". Any model built today has to beat that number to be worth anything.

Common trap. A target made of numbers is not automatically regression. Gmail's tabs could be numbered 1, 2, 3 — it is still classification, because there is no in-between value. There is no "tab 2.4".

✂ Practice 3 — classification or regression?

Q1. Ola shows you an estimated fare of Rs 247 before you book the ride.

Answer. Regression — the fare is a number in a range.

Q2. GPay flags a transaction as suspicious or not suspicious.

Answer. Classification — two labels.

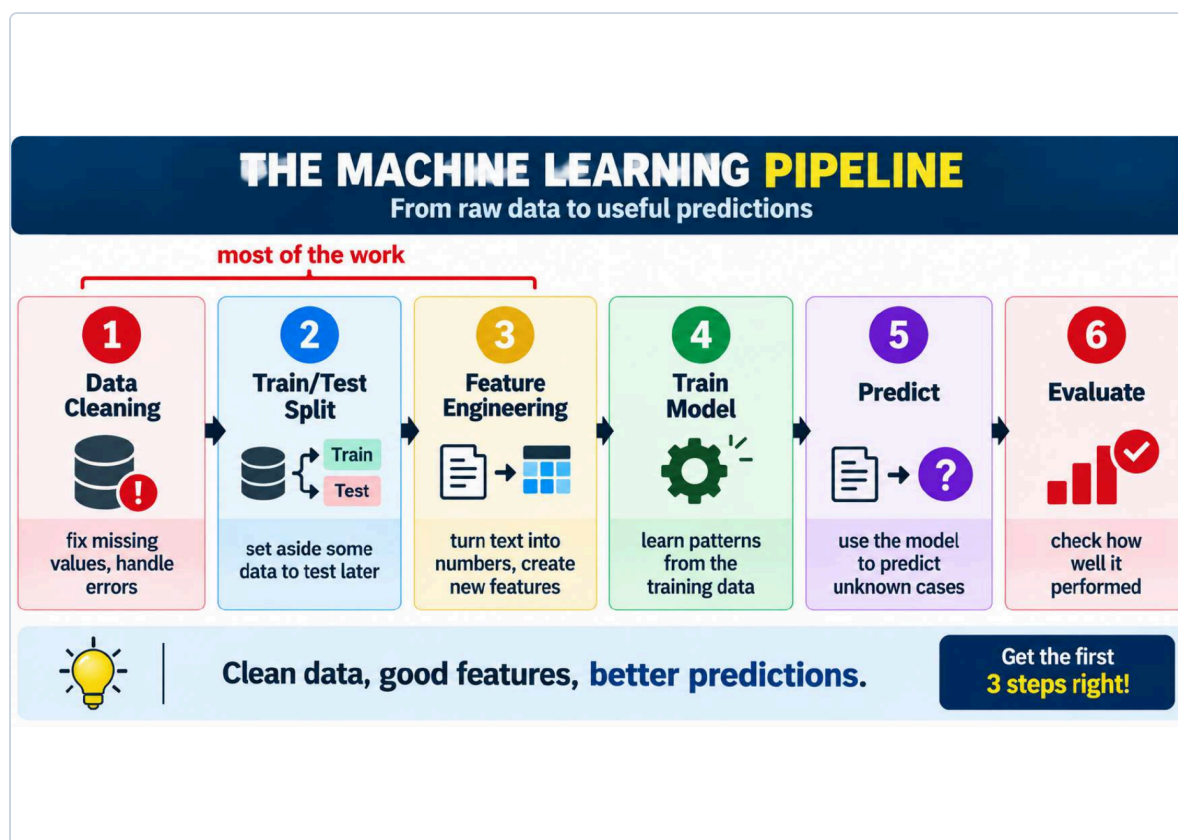
Q3. Gmail sorts an incoming mail into one of three tabs — Primary, Social or Promotions.

Answer. Classification — three labels. Numbering the tabs 1, 2, 3 would not make it regression.

4. The ML Pipeline

Flow: Six stages. Every lecture in this course sits inside one of them.

1 Clean → 2 Split → 3 Feature Engineering → 4 Train → 5 Predict → 6 Evaluate



The six stages, from raw data to a usable prediction

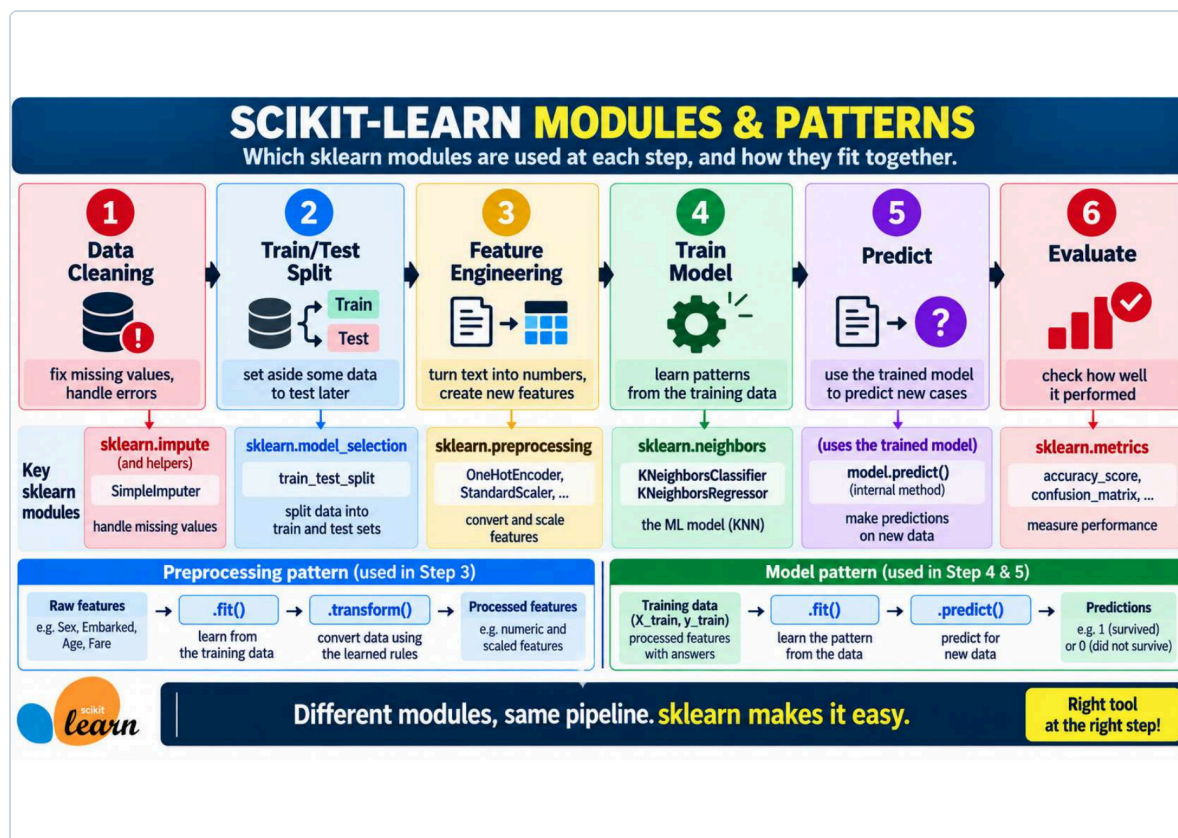
#	Stage	What happens
1	Data Cleaning	Fix missing values, drop unusable columns, handle errors
2	Train/Test Split	Set aside some rows to test on later
3	Feature Engineering	Turn text into numbers, scale, create new features
4	Train Model	Learn patterns from the training data
5	Predict	Use the trained model on unseen rows
6	Evaluate	Measure how well it actually performed

Stages 1-3 are most of the work. Training a model is often three lines of code. Getting the data into a state where those three lines work is the real job.

5. scikit-learn

Flow: One tool per stage. Two patterns.

scikit-learn (imported as `sklearn`) is the standard Python library for classical machine learning. It gives a ready-made tool for every stage of the pipeline, and — more importantly — every tool follows one of only two patterns. Learn the two patterns and every new tool becomes predictable.



Which sklearn module belongs to which stage, and the two patterns they follow

The two patterns

Type of tool	Pattern	What it means
Preprocessor OneHotEncoder, StandardScaler	<code>.fit()</code> → <code>.transform()</code>	<code>fit</code> learns the rule from the training data; <code>transform</code> applies that rule to convert the data
Model KNeighborsClassifier	<code>.fit()</code> → <code>.predict()</code>	<code>fit</code> learns the pattern; <code>predict</code> produces answers for new rows

Modules used in this lecture

Module	Used for
<code>sklearn.model_selection</code>	<code>train_test_split</code> — split data into train and test
<code>sklearn.preprocessing</code>	<code>OneHotEncoder</code> , <code>StandardScaler</code> — convert and scale features
<code>sklearn.neighbors</code>	<code>KNeighborsClassifier</code> — the model
<code>sklearn.metrics</code>	<code>accuracy_score</code> — measure performance
<code>sklearn.impute</code>	<code>SimpleImputer</code> — handle missing values

```
from sklearn.neighbors import KNeighborsClassifier
```

Note the import style: you import the specific class from the specific module, not the whole library.

6. Stage 1 — Data Cleaning

Flow: Load, find what is broken, fix it.

The question being answered: given a passenger's details, did they survive?

Look before you touch

```
df.info()  
df.head()
```

Out of 891 rows: `Age` has 714 values, `Cabin` has 204, `Embarked` has 889. Everything else is complete.

Column	Missing	Decision
Cabin	687 of 891	Drop — 77% empty, nothing left to learn from
Name	0	Drop — unique per row, so no pattern repeats
Ticket	0	Drop — same reason
PassengerId	0	Drop — just a row number
Age	177	Fill with the median — too many rows to throw away
Embarked	2	Drop those 2 rows — losing 2 of 891 costs nothing

The three fixes

```
df = df.drop(columns=["Cabin", "Name", "Ticket", "PassengerId"])
```

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df.dropna(subset=["Embarked"], inplace=True)
```

The median for `Age` is **28.0**. The median is used rather than the mean because a few very old passengers pull the mean upwards — the same reasoning used on `Fare` in the plotting lecture.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 889 entries, 0 to 890
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Survived    889 non-null    int64
1   Pclass      889 non-null    int64
2   Sex         889 non-null    object
3   Age         889 non-null    float64
4   SibSp       889 non-null    int64
5   Parch       889 non-null    int64
6   Fare        889 non-null    float64
7   Embarked    889 non-null    object
```

891 × 12 has become **889 × 8**, with zero missing values. Two columns (`Sex`, `Embarked`) are still `object` — text. Those are dealt with in Stage 3.

Rule of thumb for dropping vs filling. A column that is mostly empty gets dropped. A column with a few empty rows gets filled. A handful of broken rows in an otherwise good column can simply be deleted.

7. Features and Target

Before splitting anything, separate what the model is given from what it has to predict.

```
x = df.drop(columns=["Survived"]) # features – what the model sees
y = df["Survived"]               # target – what the model must predict
```

Name	Shape	Contents
x	889 × 7 (DataFrame)	Pclass, Sex, Age, SibSp, Parch, Fare, Embarked
y	889 (Series)	Survived — 0 or 1

Naming convention: lowercase `x` and `y` here; you will also see capital `X` in most documentation, because the features form a 2-D table while the target is a single column.

8. Stage 2 — Train/Test Split

Flow: Hide some rows before preparing anything.

A model tested on the same rows it learned from will look far better than it really is — it has already seen the answers. So a portion of the data is hidden away before any further work happens, and is only brought back at the very end to check the model honestly.

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
```

The function returns **four** objects, always in this order: train features, test features, train target, test target.

Object	Shape	Used for
x_train	711 × 7	Learning
x_test	178 × 7	Final check only
y_train	711	The answers the model learns from
y_test	178	The answers kept hidden until evaluation

The arguments

Argument	What it does
<code>test_size=0.2</code>	20% of rows go to the test set. 0.2 or 0.25 is standard.
<code>random_state=42</code>	Fixes which rows are picked, so everyone in the class gets the identical split and identical results. Remove it and the numbers change on every run.
<code>stratify=y</code>	Keeps the same survival ratio in both halves. Important when one class is rarer than the other.

The rule that causes the most mistakes: the split happens **before** encoding and scaling, not after. Anything learned from the data — a median, a set of categories, a mean and standard deviation — must be learned from the training rows only. If the test rows influence that learning, the final score is inflated and meaningless.

9. Stage 3 — Encoding

Flow: Turn text columns into numbers.

scikit-learn works on numbers only. `Sex` holds "male"/"female" and `Embarked` holds "S"/"C"/"Q", so both have to be converted before any model can be trained. Which conversion to use depends on whether the categories have a natural order.

ENCODING CATEGORICAL FEATURES

Different techniques for different types of categorical data

1 NOMINAL ENCODING
For categories with no natural order

Feature: Embarked	
S	(Southampton)
C	(Cherbourg)
Q	(Queenstown)

One-Hot Encoding (new columns)		
Embarked_S	Embarked_C	Embarked_Q
1	0	0
0	1	0
0	0	1

- Creates a new column for each category
- No order is assumed

2 ORDINAL ENCODING
For categories with a natural order

Feature: Pclass	
1	(First class)
2	(Second class)
3	(Third class)

Ordinal Encoding	
Pclass	Encoded Value
1	1
2	2
3	3

- Preserves the natural order between categories
- Useful when order matters

3 LABEL ENCODING
For the target column only

Target: Survived	
yes	
no	

Label Encoding	
Survived	Encoded Value
yes	1
no	0

- y = 'yes' / 'no' → 0, 1
- for the target column only

❌ DON'T DO THIS
Don't use label encoding for nominal features!

Embarked
S
C
Q

~~Embarked = S, C, Q → 0, 1, 2~~

- This incorrectly gives an order to categories.
- Use one-hot encoding instead.

 Choose the **right encoding technique** for your data.

Better features lead to better models!

Three encoding types, and the one mistake to avoid

Type	Use when	Example
Nominal one-hot encoding	Categories have no order	<code>Embarked</code> S/C/Q → three 0/1 columns
Ordinal ordinal encoding	Categories have a real order	<code>Pclass</code> 1 < 2 < 3 — already numbered correctly
Label label encoding	Target column only	<code>Survived</code> yes/no → 1/0

Do not do this. Encoding `Embarked` as S=0, C=1, Q=2 tells the model that Q is greater than C, and that C sits exactly halfway between S and Q. None of that is true — these are just three ports. Ordering categories that have no order invents a relationship that does not exist. Use one-hot instead.

One-hot encoding in code

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False)
encoder.fit(x_train[["Sex", "Embarked"]])    # learn the categories – training rows only
```

Argument	What it does
<code>sparse_output=False</code>	Returns a normal NumPy array instead of a sparse matrix, so the output can be read and joined back easily
<code>handle_unknown='ignore'</code>	If a category appears in the test set that was never seen during <code>fit</code> , output all zeros instead of raising an error
<code>drop='first'</code>	Drops one column per feature to avoid duplicated information — optional, and not needed for KNN

The two-pattern rule from Section 5 applies here exactly: `fit` on the training rows to learn which categories exist, then `transform` to convert.

```
encoder.categories_
# [array(['female', 'male']), array(['C', 'Q', 'S'])]

encoded = encoder.transform(x_train[["Sex", "Embarked"]])
encoded.shape
# (711, 5)
```

Two text columns have become five numeric columns:

Sex_female	Sex_male	Embarked_C	Embarked_Q	Embarked_S
1	0	0	0	1
1	0	1	0	0
0	1	0	0	1

Each row has exactly one 1 per original column — hence the name "one-hot". A female passenger who boarded at Southampton is `1, 0, 0, 0, 1`.

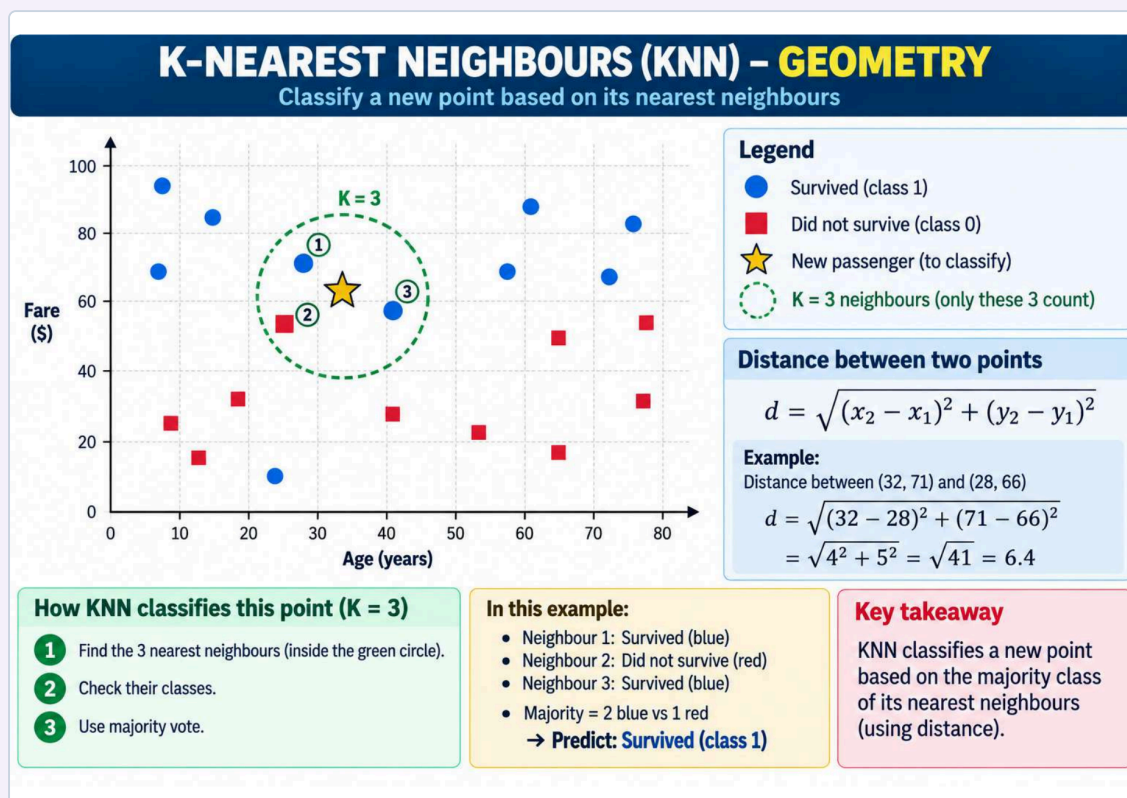
Fit on train, transform on both. The encoder is fitted only on `x_train`. The same fitted encoder is then used to `transform` both `x_train` and `x_test`. Never call `fit` again on the test data — that would let the test set influence the preparation.

Coming next class

Two ideas were introduced with diagrams and will be built in code next session.

K-Nearest Neighbours — the first model

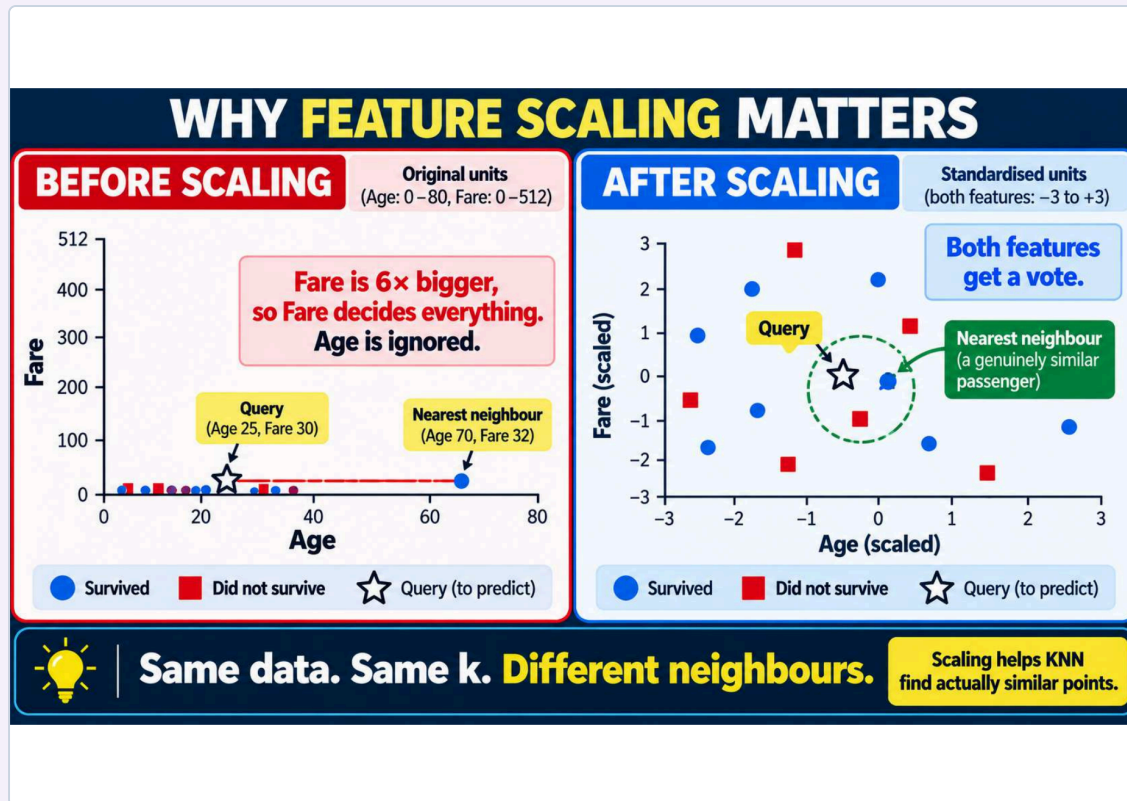
KNN does not really "learn" a formula. To classify a new passenger, it measures the distance to every passenger in the training data, takes the *k* closest ones, and gives the new passenger whichever answer the majority of those neighbours had.



With *k* = 3, two of the three nearest neighbours survived, so the prediction is "survived"

Why scaling is needed before KNN

KNN is built entirely on distance. Age runs 0–80 while Fare runs 0–512, so a difference in Fare counts roughly six times more than the same difference in Age, and Age is effectively ignored. Putting both columns on the same scale gives every feature a vote.



Same data, same k, different neighbours — scaling changes who is considered "closest"

Quick Reference

Everything covered today, in one block

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

# Stage 1 – clean
df = pd.read_csv('Titanic-Dataset.csv')
df = df.drop(columns=["Cabin", "Name", "Ticket", "PassengerId"])
df["Age"] = df["Age"].fillna(df["Age"].median())
df.dropna(subset=["Embarked"], inplace=True)

# features and target
x = df.drop(columns=["Survived"])
y = df["Survived"]

# Stage 2 – split
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)

# Stage 3 – encode
encoder = OneHotEncoder(sparse_output=False)
encoder.fit(x_train[["Sex", "Embarked"]])
encoded_train = encoder.transform(x_train[["Sex", "Embarked"]])
encoded_test = encoder.transform(x_test[["Sex", "Embarked"]])
```

Decision checklist for any new dataset

1. Is there an answer column? → supervised, else unsupervised
2. Is the target a number in a range or a fixed label? → regression or classification
3. What is missing, and is it worth filling or dropping?
4. Split before preparing anything
5. Which text columns need encoding, and do their categories have an order?

Common errors

What you see	Why, and the fix
could not convert string to float	A text column reached the model. Encode <code>Sex</code> and <code>Embarked</code> first.
Input contains NaN	Cleaning was skipped or incomplete. Check <code>df.isna().sum()</code> .
Accuracy changes on every run	<code>random_state</code> was not set in <code>train_test_split</code> .
Found unknown categories during transform	A category exists in test but not in train. Use <code>handle_unknown='ignore'</code> .
Suspiciously high accuracy	The test data influenced training. Split first, and <code>fit</code> only on train.
ValueError: Expected 2D array	A single column was passed as a Series. Use double brackets: <code>x_train[["Sex"]]</code> .

Glossary

Term	Meaning
Feature	An input column the model is allowed to see (<code>x</code>)
Target / label	The column being predicted (<code>y</code>)
Labelled data	Data where the answer column exists
Training set	The rows the model learns from
Test set	The hidden rows used only to judge the model
Baseline	The score obtained by always guessing the most common class — 61.6% here
<code>fit</code>	Learn something from the training data
<code>transform</code>	Apply what was learned to convert data
<code>predict</code>	Produce answers for new rows
One-hot encoding	One new 0/1 column per category, no order implied